

Recursividad (con funciones auxiliares)

Programación I - UNGS

Antes de empezar...

¿Cuál es la diferencia entre un **problema** y una **instancia de un problema**?

Antes de empezar...

¿Cuál es la diferencia entre un **problema** y una **instancia de un problema**?

- Ejemplos de problemas:
 - Hallar la cantidad de divisores de **un número**.
 - Decidir si **un número** es o no primo.
 - Contar la cantidad de apariciones de **un caracter** en **una cadena**.

Antes de empezar...

¿Cuál es la diferencia entre un **problema** y una **instancia de un problema**?

- Ejemplos de problemas:
 - Hallar la cantidad de divisores de **un número**.
 - Decidir si **un número** es o no primo.
 - Contar la cantidad de apariciones de **un caracter** en **una cadena**.
- Ejemplos de instancias de problemas:
 - Hallar la cantidad de divisores **del número 20**.
 - Decidir si **el número 2683** es o no primo.
 - Contar la cantidad de apariciones de **la letra 'a'** en **la cadena "Hola mundo"**.

Repasemos recursividad...

Para escribir un método recursivo tenemos que:

Repasemos recursividad...

Para escribir un método recursivo tenemos que:

1. Identificar los **casos base**.

Repasemos recursividad...

Para escribir un método recursivo tenemos que:

1. Identificar los **casos base**.
2. Resolver una **instancia grande** en función de **instancias más chicas** del mismo problema (las cuales resolvemos con llamados recursivos).

Repasemos recursividad...

Para escribir un método recursivo tenemos que:

1. Identificar los **casos base**.
2. Resolver una **instancia grande** en función de **instancias más chicas** del mismo problema (las cuales resolvemos con llamados recursivos).

Pregunta: ¿Siempre podemos resolver una instancia de un problema en función de instancias más chicas del mismo problema?

Ejemplo

Escribir un método (usando recursión) para calcular la cantidad de divisores de un número, con el siguiente encabezado:

1 **static int** cantDivisores(**int** n)

- ¿Cuáles/cuáles son los **casos base**?

Ejemplo

Escribir un método (usando recursión) para calcular la cantidad de divisores de un número, con el siguiente encabezado:

1 **static int** cantDivisores(**int** n)

- ¿Cuáles/cuáles son los **casos base**?
- Si me piden cantDivisores(n), ¿qué sería una instancia “más chica” de este problema?

Ejemplo

Escribir un método (usando recursión) para calcular la cantidad de divisores de un número, con el siguiente encabezado:

1 **static int** cantDivisores(**int** n)

- ¿Cuáles/cuáles son los **casos base**?
- Si me piden cantDivisores(n), ¿qué sería una instancia “más chica” de este problema?
- ¿Me sirve eso para resolver la instancia cantDivisores(n)?

Funciones auxiliares

- Muchas veces no es posible resolver una instancia de un problema en función de instancias más chicas.

Funciones auxiliares

- Muchas veces no es posible resolver una instancia de un problema en función de instancias más chicas.
- Pero siempre se puede **reescribir el problema** a otro **problema equivalente (o más general)**.

Funciones auxiliares

- Muchas veces no es posible resolver una instancia de un problema en función de instancias más chicas.
- Pero siempre se puede **reescribir el problema** a otro **problema equivalente (o más general)**.
- Resolvemos luego el nuevo problema usando recursión y... *voilà!*

Funciones auxiliares

- Muchas veces no es posible resolver una instancia de un problema en función de instancias más chicas.
- Pero siempre se puede **reescribir el problema** a otro **problema equivalente (o más general)**.
- Resolvemos luego el nuevo problema usando recursión y... *voilà!*
- El problema es que no siempre está claro cómo hacer esto... :-)

Volviendo al ejemplo de `cantDivisores(n)`

- Para [este problema](#), necesitamos contar los divisores de n y los candidatos son los números en el intervalo $[1, n]$.

Volviendo al ejemplo de cantDivisores(n)

- Para **este problema**, necesitamos contar los divisores de n y los candidatos son los números en el intervalo $[1, n]$.
- Podemos **transformar el problema** en otro parecido que cuente los divisores de un número dentro de un determinado rango:

```
1 static int cantDivisoresEnRango(int n, int d, int h)
```

¿Podemos resolver este problema? Asumamos que sí...

Volviendo al ejemplo de cantDivisores(n)

- Para **este problema**, necesitamos contar los divisores de n y los candidatos son los números en el intervalo $[1, n]$.
- Podemos **transformar el problema** en otro parecido que cuente los divisores de un número dentro de un determinado rango:

```
1 static int cantDivisoresEnRango(int n, int d, int h)
```

¿Podemos resolver este problema? Asumamos que sí...

- ¿Nos sirve esta **función auxiliar** para resolver el problema original?

```
1 static int cantDivisores(int n)
2 {
3     return ???;
4 }
```

Volviendo al ejemplo de cantDivisores(n)

- Para **este problema**, necesitamos contar los divisores de n y los candidatos son los números en el intervalo $[1, n]$.
- Podemos **transformar el problema** en otro parecido que cuente los divisores de un número dentro de un determinado rango:

```
1 static int cantDivisoresEnRango(int n, int d, int h)
```

¿Podemos resolver este problema? Asumamos que sí...

- ¿Nos sirve esta **función auxiliar** para resolver el problema original?

```
1 static int cantDivisores(int n)
2 {
3     return cantDivisoresEnRango(n, 1, n);
4 }
```

Volviendo al ejemplo de cantDivisores(n)

Ahora tenemos que resolver la función auxiliar:

```

1 static int cantDivisoresEnRango(int n, int d, int h)
2 {
3     ???
4 }
```

Volviendo al ejemplo de cantDivisores(n)

Ahora tenemos que resolver la función auxiliar:

```

1 static int cantDivisoresEnRango(int n, int d, int h)
2 {
3     ???
4 }
```

- ¿Cuáles son los **casos base**?

Volviendo al ejemplo de cantDivisores(n)

Ahora tenemos que resolver la función auxiliar:

```

1 static int cantDivisoresEnRango(int n, int d, int h)
2 {
3     ???
4 }
```

- ¿Cuáles son los **casos base**?
- ¿Cómo resolvemos un caso no base?

Volviendo al ejemplo de cantDivisores(n)

Ahora tenemos que resolver la función auxiliar:

```

1 static int cantDivisoresEnRango(int n, int d, int h)
2 {
3     if (d > h)
4         return 0;
5
6     if (n % d == 0)
7         return 1 + cantDivisoresEnRango(n, d+1, h);
8     else
9         return cantDivisoresEnRango(n, d+1, h);
10 }
```

Volviendo al ejemplo de cantDivisores(n)

Ahora tenemos que resolver la función auxiliar:

```

1 static int cantDivisoresEnRango(int n, int d, int h)
2 {
3     if (d > h)
4         return 0;
5
6     return (n % d == 0? 1 : 0) + cantDivisoresEnRango(n, d+1, h);
7 }
```

En resumen

- A veces no es posible (o no es fácil) resolver una instancia grande en función de instancias más chicas **del mismo problema**.

En resumen

- A veces no es posible (o no es fácil) resolver una instancia grande en función de instancias más chicas **del mismo problema**.
- Pero siempre podemos transformar el problema en **otro más general** y resolver el original por medio de éste.

En resumen

- A veces no es posible (o no es fácil) resolver una instancia grande en función de instancias más chicas **del mismo problema**.
- Pero siempre podemos transformar el problema en **otro más general** y resolver el original por medio de éste.
- Para que esto tenga sentido, una instancia grande del nuevo problema debería poder resolverse en función de instancias más chicas de ese mismo problema.

En resumen

- A veces no es posible (o no es fácil) resolver una instancia grande en función de instancias más chicas **del mismo problema**.
- Pero siempre podemos transformar el problema en **otro más general** y resolver el original por medio de éste.
- Para que esto tenga sentido, una instancia grande del nuevo problema debería poder resolverse en función de instancias más chicas de ese mismo problema.
- Lamentablemente, esta metodología no es “automática” y requiere práctica... :-)